

COS 710: Artificial Intelligence

Assignment 3: Structure-Based Grammatical Evolution for Regression

Yohali Malaika Kamangu

u23618583

May 2026

Dataset: Residential Energy Usage UK (2014–2020)

0. Contents

1	Description of the Data Set	2
2	Description of the Performance Metric	2
3	How Structure is Incorporated into Canonical GE	2
4	Description of the Grammar Used	3
5	Initial Population Generation	4
6	Fitness Function and Fitness Evaluation	5
7	Selection Method	5
8	Genetic Operators	5
9	Experimental Setup	6
9.1	Hardware and software environment	6
9.2	Parameter values	7
9.3	Reproducibility	7
10	Results and Discussion	8
10.1	Aggregated results across 10 runs	8
10.2	Per-run results	8
10.3	Best evolved expression	8
10.4	Per-generation behaviour of the best run	9
10.5	Discussion	9
11	Runtimes	10
12	Comparison with Assignments 1 and 2	10
12.1	Quality of solutions	10
12.2	Robustness	11
12.3	Speed	11
12.4	Summary	11
13	References	11

1. Description of the Data Set

The data set is the Residential Energy Usage United Kingdom data set covering the years 2014–2020. Each row in the raw CSV is one observation sampled at 15-minute intervals and includes the columns `utc_timestamp`, `Electricity_load`, `Residential_electricity_price`, `Residential_solar_generation`, `Residential_wind_generation`, `Temperature` and `Relative Humidity`. Only the `Electricity_load` column is used in this assignment, in keeping with Assignments 1 and 2: the task is defined as a univariate time-series regression problem in which the target y_t is the load value at time t and the inputs are the seven most recent observed lag values $(y_{t-1}, y_{t-2}, \dots, y_{t-7})$.

To keep run-times tractable but the data sample large enough for stable statistics, the first 100,000 valid rows of the CSV are read. A sliding window of size 7 is then applied to produce 99,993 supervised examples. These are split temporally (no shuffling) into 79,994 training and 19,999 test examples (80/20). Temporal ordering is preserved because shuffling a time-series would leak future information into the training set.

2. Description of the Performance Metric

Mean Squared Error (MSE) is used as the primary measure of predictor quality:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad (1)$$

where $\hat{y}_i$ is the value produced by the evolved expression on the i -th windowed input and y_i the actual measured load. MSE is chosen for three reasons: (i) it is differentiable and convex, providing a smooth selection pressure surface; (ii) it penalises large errors quadratically, which matches the operational cost of grid-load mis-prediction; (iii) it is identical to the metric used in Assignments 1 and 2, ensuring that the three sets of results are directly comparable.

Two secondary indicators are also reported: the *hits count* (training cases with $|\hat{y}_i - y_i| < 0.01$) and the *success predicate* (a run is considered successful if its training MSE is below 2×10^{-4} , the same threshold used in A1 and A2).

3. How Structure is Incorporated into Canonical GE

Canonical Grammatical Evolution [1] represents each individual as a variable-length integer codon genome which is mapped through a BNF grammar to a phenotype program. Selection, crossover and mutation operate on the genome (the genotype); fitness is computed on the phenotype. Canonical GE has no explicit structural component: every behaviourally good genome is selected with the same probability, even when the population has collapsed onto a single derivation skeleton.

In this work canonical GE is extended with a *structural fitness-sharing* penalty applied at the derivation level. During mapping the sequence of (non-terminal, production) pairs chosen by the leftmost derivation is recorded as a *derivation signature* — a compact representation of the program’s structural skeleton that is independent of its constant values. Two individuals are considered to occupy the same structural niche when their

normalised prefix-match ratio exceeds a threshold σ . The raw fitness is then inflated by a multiplicative penalty proportional to the niche count:

$$f_{\text{adj}}(i) = f_{\text{raw}}(i) \left(1 + \alpha \cdot \text{nicheCount}(i)\right) \quad (2)$$

with $\alpha = 0.01$ and $\sigma = 0.6$ used in this work. The niche count is estimated from 50 uniformly sampled peers per individual rather than the full population to keep the per-generation cost linear in the population size.

Why derivation signatures and not phenotype trees. The structural sharing in Assignment 2 used prefix matching on the GP expression *tree*. In GE the equivalent natural object is the derivation tree, but the derivation tree carries strictly more information than the phenotype tree (it records which grammar production was chosen even when several productions lead to the same phenotype, e.g. productions 0 and 1 of $\langle \text{expr} \rangle$). This makes the GE signature a finer-grained structural measure — two individuals are only considered similar if the grammar walked exactly the same path, which is the right notion of similarity for a grammar-based search.

Elitism uses raw, not adjusted, fitness. A subtle but important design point (highlighted as feedback on Assignment 2): the goal of elitism is to guarantee that the best *behavioural* solution discovered so far is carried unchanged into the next generation. Whether that individual happens to share its structural skeleton with many peers is irrelevant to its predictive quality, so the elite slot is chosen by raw MSE-based fitness. The adjusted fitness in Equation 2 only influences *parent selection*, which is where structural diversity needs to be encouraged.

4. Description of the Grammar Used

A compact BNF grammar suitable for closed-form symbolic regression on the seven lag features is used:

```

<expr> ::= <expr> <op> <expr>           [production 0]
         | ( <expr> <op> <expr> )         [production 1]
         | <var>                          [production 2]
         | <const>                        [production 3]
<op>    ::= + | - | * | /
<var>   ::= x1 | x2 | x3 | x4 | x5 | x6 | x7
<const> ::= <digit>.<digit><digit>       [single production]
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

Why these productions. The $\langle \text{expr} \rangle$ non-terminal has four productions chosen to give the grammar both recursive expressive power (productions 0 and 1) and a terminating route (productions 2 and 3). Including the parenthesised production alongside the bare one slightly biases the derivation toward sub-expressions that are clearly delimited; functionally the two yield the same AST shape but doubling the count of recursive options means a random codon is twice as likely to grow the tree as to terminate it, balancing the two terminating productions and producing a healthy depth distribution in the initial population.

Why these terminals (the seven-lag input set). The terminal set $\{x_1, \dots, x_7\}$ corresponds to the seven most recent observed load values $(y_{t-1}, \dots, y_{t-7})$. Seven lags were chosen for three concrete reasons. First, the residential energy data is sampled at 15-minute intervals, so seven lags cover the most recent 1 h 45 min of load — the sub-hourly window over which the auto-correlation function of the series is strongest. Second, a small autocorrelation pilot study performed for Assignment 1 on the same series showed that the seven-lag window captures over 95% of the linear predictive power and that adding further lags produced negligible gains. Third, using the same terminal set as A1 and A2 keeps the search space identical across the three assignments, so any performance differences can be attributed to the search engine rather than to the input encoding.

Why include ephemeral random constants. The $\langle \text{const} \rangle$ production yields three-digit ERCs in $[0.00, 9.99]$. Without ERCs the grammar can only construct expressions whose numerical value comes from input variables, which makes it impossible to represent simple but useful transformations such as *averaging* the two most recent lags (which requires the constant 2, or the equivalent 1.99 produced by the best run of this assignment). ERCs are therefore essential for the closure of the representable function space under arithmetic scaling and offsetting. The constant range $[0.00, 9.99]$ was chosen empirically: it covers the small scaling factors typical of normalised time-series averaging without permitting wildly large constants that destabilise selection.

Termination guarantees. Two safeguards prevent the mapping from looping forever. (a) The codon stream is allowed to *wrap* at most twice; a third wrap declares the genome invalid and assigns it the worst fitness so it is removed by selection. (b) Once the derivation depth equals six — the same depth cap used for GP trees in A1/A2 — $\langle \text{expr} \rangle$ is restricted to its terminal productions ($\langle \text{var} \rangle$ or $\langle \text{const} \rangle$). This is the depth-bounded mapping used in standard GE implementations such as PonyGE2.

5. Initial Population Generation

Each genome in the initial population is constructed by drawing 64 uniform random integers in $[0, 256)$. This is the canonical GE initialisation [1].

Why random codon initialisation and not sensible initialisation. GE has two well-known initialisation strategies: *random codon initialisation* (used here) and *sensible initialisation* (Azad and Ryan, 2003), which is the GE analogue of GP’s ramped half-and-half: it grows the derivation tree to controlled depths and back-solves for the codons needed to produce it. Sensible initialisation guarantees that no individual in generation 0 is invalid or trivially small, which is its main advantage over random initialisation.

Random codon initialisation was preferred in this work for three reasons. (1) *Structural diversity for sharing.* The structural fitness-sharing penalty (Section 3) depends on a heterogeneous initial population: if every genome already shares a similar derivation skeleton (as sensible initialisation tends to produce because it draws depths from a small ramp) the sharing penalty has little to push against in the early generations. Pure random codon initialisation guarantees a wide spread of derivation signatures from generation 0. (2) *Termination is already guaranteed.* The depth-bounded fallback in the grammar (Section 4) ensures every random codon stream maps to a complete sentence within the

depth cap, removing the main motivation for sensible initialisation. (3) *Simplicity / parity with the literature*. Random codon initialisation is the canonical choice in the original O’Neill and Ryan formulation and the most common choice in published GE benchmark studies, making the results easier to compare.

The genome length of 64 codons was chosen as a conservative upper bound on the number of derivation steps required by an expression at the maximum depth of 6: a balanced binary $\langle \text{expr} \rangle$ tree of depth 6 needs at most $2^6 - 1 = 63$ rule choices. A 64-codon genome therefore typically maps without wrapping, which makes the genotype to phenotype mapping easier to reason about and limits the disruptive effect of crossover on long genomes.

6. Fitness Function and Fitness Evaluation

Raw fitness is the parsimony-adjusted training MSE:

$$f_{\text{raw}}(i) = \text{MSE}(i) (1 + c \cdot s(i)) \quad (3)$$

where $s(i)$ is the number of nodes in the phenotype AST and $c = 10^{-3}$ a small parsimony coefficient. Parsimony pressure is identical to that used in A1 and A2 and is included because GE, like GP, suffers from bloat: in early experiments without parsimony the mean phenotype size grew unboundedly after the structural sharing penalty spread the population across many niches. Individuals whose mapping is incomplete (more than `MAX_WRAPS` wraps) or whose evaluation produces NaN or infinity are assigned $f_{\text{raw}} = \infty$, which eliminates them from selection.

The adjusted (shared) fitness used for *parent selection only* is given by Equation 2. The elite slot — one slot per generation — is filled by the individual with the lowest raw fitness, not the lowest adjusted fitness.

Evaluation is purely deterministic and runs the phenotype interpreter once per training example. Protected division is used: if the denominator has absolute value below 10^{-9} the operator returns 1 rather than producing a division-by-zero.

7. Selection Method

Parent selection is tournament selection of size 2 over the adjusted fitness. Two individuals are sampled with replacement and the one with lower adjusted fitness is selected. The tournament size was kept at 2 for direct parity with A1 and A2; the same low selection pressure preserves population variety long enough for the structural sharing penalty to have a measurable effect.

8. Genetic Operators

Three operators act on the population each generation: *elitism*, *one-point crossover* on integer codon arrays, and *integer mutation* per codon. Operator application uses an *exact-rate split* rather than per-child Bernoulli probabilities: given a population of size N , exactly one slot is filled by elitism, exactly $\lfloor 0.70(N - 1) \rfloor$ slots are produced by crossover, and the remaining $\lfloor 0.30(N - 1) \rfloor$ slots are produced by mutation. The crossover and mutation rates therefore sum to 1.0. Each operator is justified individually below.

Elitism (one slot per generation). The single best individual by *raw* fitness is copied unchanged into the next generation. This guarantees the monotone improvement of the best-so-far behavioural solution from one generation to the next — the destabilising effect of crossover and mutation can otherwise destroy a hard-earned good individual when the structural sharing penalty has demoted it because its niche has become crowded. As noted in Section 3, raw rather than adjusted fitness is used here because elitism is about preserving the best *behavioural* solution.

One-point crossover on codons (rate 0.70). A single crossover point is sampled uniformly along the 64-codon genome; the child inherits codons before the cut from parent 1 and codons after the cut from parent 2. This is the standard recombination operator of canonical GE [1]. It is included because in a leftmost derivation the early codons control the structural choices made high in the derivation tree — the program’s skeleton — while later codons refine sub-expressions and constants. A one-point cut therefore cleanly exchanges the *skeleton* of one parent with the *refinement* of another, which is exactly the recombination of building blocks that drives improvement in GE. Crossover is included as the principal *exploitation* operator: it produces new individuals entirely from material that has already survived selection, which is the mechanism by which the search consolidates partial solutions into stronger ones.

Integer mutation (per-codon probability 0.10). Thirty percent of children come from mutation. The mutation operator takes one selected parent and produces a child in which each codon is independently replaced with probability 0.10 by a fresh uniform draw in $[0, 256)$. Mutation is included as the principal *exploration* operator: because a single codon change can swap an entire production choice (and therefore replace one sub-expression skeleton with another), integrating mutation into 30% of the new population keeps the search from collapsing onto whatever skeleton the elite currently uses. In pilot runs with the operator split set to 0.9/0.1 the search consistently converged onto the trivial predictor $\hat{y} = y_{t-1}$ in six of ten seeds; restoring the 0.7/0.3 balance yielded a more diverse spread of final solutions, confirming the role of mutation as the exploratory driver.

Together the three operators implement a balanced exploitation/exploration cycle: elitism preserves the best behaviour, crossover recombines existing structural building blocks, and mutation seeds the population with previously unseen production choices.

9. Experimental Setup

9.1 Hardware and software environment

All simulations were executed on the following machine:

- Apple MacBook Pro, M-series ARM64 CPU
- macOS 14
- Oracle OpenJDK 17 (single-threaded)
- No external libraries; only the Java standard library is used

9.2 Parameter values

The GE control parameters were chosen as follows. Where a parameter also exists in A1/A2 it was kept identical to allow direct comparison of the search engines themselves; structural sharing parameters were inherited from A2 because they were already validated there.

Parameter	Value	Justification
Population size	1000	Matches A1/A2
Generations	100	Matches A1/A2
Tournament size	2	Low pressure preserves variety
Crossover rate	0.70	Exploitation operator (see Sec. 8)
Mutation rate	0.30	Exploration operator; $0.70 + 0.30 = 1.0$
Per-codon mutation prob.	0.10	≈ 6 codons/genome on average
Genome length	64	Upper bound on derivation steps at depth 6
Codon range	$[0, 256)$	Standard 8-bit codons
Max derivation depth	6	Matches A1/A2 GP depth cap
Max wraps	2	Standard GE value
Window size (lags)	7	Matches A1/A2
Train/test ratio	0.80	Temporal split (no shuffling)
σ (similarity thresh.)	0.60	Matches A2
α (sharing coeff.)	0.01	Matches A2
Structural sample size	50	Linear-time niche estimate
Parsimony coefficient	10^{-3}	Matches A1/A2
Independent runs	10	Seeds 101, 202, $\dots$, 1010

Table 1: SBGE control parameters used in all experiments.

Two control parameters were tuned for this assignment: the operator split and the per-codon mutation probability. The operator split was first set to 0.9/0.1 on the basis that crossover is GE’s principal recombination operator; however, six of ten pilot runs converged onto the trivial $\hat{y} = y_{t-1}$ predictor under that setting because mutation-free children dominated the new population and the elite rapidly homogenised the population through tournament selection. Resetting the split to 0.7/0.3 removed this collapse mode and was adopted. The per-codon probability was then set to 0.10 (≈ 6 codon changes per mutated child) to give a meaningful structural perturbation while keeping the average derivation-signature change below half the genome. The remaining hyper-parameters were inherited from A1/A2 and not re-tuned.

9.3 Reproducibility

The same set of ten seeds $\{101, 202, 303, 404, 505, 606, 707, 808, 909, 1010\}$ is used as in A1 and A2. With identical seeds, the same random codon sequence is generated, so any reader who recompiles the source code on a JDK 17 runtime will obtain the exact numerical results reported in Section 10.

10. Results and Discussion

10.1 Aggregated results across 10 runs

Statistic	Train MSE	Test MSE
Mean over 10 runs	1.9314×10^{-4}	2.3663×10^{-4}
Standard deviation	2.4060×10^{-5}	—
Best run (Run 4, seed 404)	1.6529×10^{-4}	1.9213×10^{-4}

Table 2: Summary statistics for SBGE across the ten independent runs.

10.2 Per-run results

Run	Train MSE	Test MSE	Time (s)
1	0.00018336	0.00021950	20.89
2	0.00021701	0.00026954	23.45
3	0.00016685	0.00020368	31.22
4	0.00016529	0.00019213	39.77
5	0.00021701	0.00026954	19.69
6	0.00021607	0.00026763	21.80
7	0.00016588	0.00020372	60.86
8	0.00021701	0.00026954	33.53
9	0.00016656	0.00020373	69.53
10	0.00021639	0.00026731	45.10
Mean	0.00019314	0.00023663	36.58

Table 3: Per-run results across the ten seeds. Best run bolded.

10.3 Best evolved expression

The best individual found across all ten runs was produced by Run 4 (seed 404):

$$\hat{y}_t = \frac{2.92 x_7}{4.48} + \frac{x_4}{1.75 (x_7 + 0.10) x_7 \left(\frac{8.08}{x_3} + 8.86 \right)}.$$

The expression combines a scaled y_{t-7} baseline with a small residual correction driven by y_{t-3} and y_{t-4} . Although less interpretable than a pure moving average, its test MSE of 1.92×10^{-4} is the closest any SBGE run came to the Assignment 1 baseline.

10.4 Per-generation behaviour of the best run

Gen	Avg Train MSE	Avg Size	Hits	Variety	Struct. Sim.
10	1.64×10^3	1.75	39 240	49	0.46
20	1.22×10^3	2.89	39 088	103	0.37
30	1.44×10^2	3.78	39 088	122	0.09
40	8.45×10^7	4.68	39 161	245	0.09
50	5.43×10^4	5.61	39 161	334	0.05
60	2.09×10^4	5.33	41 903	372	0.11
70	5.81×10^2	5.39	42 258	381	0.11
80	3.22×10^3	5.49	42 258	388	0.08
90	3.95×10^6	5.23	42 852	433	0.05
100	1.18×10^3	5.28	42 852	435	0.12

Table 4: Per-generation statistics for the best run (Run 4, seed 404). “Variety” is the number of distinct phenotype expressions in the population; “Struct. Sim.” is the mean pairwise normalised prefix match over the derivation signatures of 50 sampled pairs.

10.5 Discussion

Convergence behaviour. The hits count climbs from 39,240 (49% of the training set) at generation 10 to 42,852 (53.6%) by generation 100, while the average phenotype size grows from 1.75 to about 5.3. Hits saturate well before the MSE saturates, indicating that once the search has found a structurally adequate skeleton further refinement happens through small adjustments of the embedded constants rather than through changes to the shape of the expression.

Why the average MSE is so large. The reported *average* training MSE per generation is enormous because it is dominated by a tail of individuals whose evolved expression contains an unfortunate combination of multiplications and mid-range constants. Such phenotypes produce outputs in the thousands or millions, even though they are valid programs and therefore counted in the average. The structural sharing penalty encourages the population to maintain such structurally distinct individuals — which is exactly the design intent — but it inflates the per-generation mean. The robust measures (best-so-far raw fitness, hits) tell the real story: the search consistently improves its best individual generation by generation.

Effect of fitness sharing on diversity. The average pairwise structural similarity falls steadily from 0.46 at generation 10 to between 0.05 and 0.12 from generation 50 onwards, and phenotype variety rises from 49 to over 400. Compared with the equivalent figures recorded for the canonical Assignment 1 GP run (variety collapsed to the low hundreds by generation 100), the sharing penalty is clearly maintaining a wider structural spread throughout the search. This is the expected qualitative behaviour of fitness-sharing in any evolutionary algorithm, and it is reproduced here on derivation signatures rather than on tree topology.

Success predicate. Five of the ten runs reach the success threshold of $\text{MSE} < 2 \times 10^{-4}$. The remaining five runs (2, 5, 6, 8, 10) all converge to the same plateau ($\text{MSE} \approx 2.17 \times 10^{-4}$),

which corresponds to the trivial single-lag predictor $\hat{y} = y_{t-1}$. Inspection of those runs’ per-generation statistics shows that they exhausted the early structural diversity before the sharing penalty could move them off this attractor: in GE, as in canonical GP, the most recent lag is a deep local optimum, and a stronger exploration mechanism (e.g. random restarts or grammar immunisation) would likely be required to consistently escape it.

11. Runtimes

The average wall-clock time per run across the ten seeds was 36.58 s, with a minimum of 19.69 s (Run 5) and a maximum of 69.53 s (Run 9). The total wall-clock time for all ten runs was approximately 6 min 06 s. The variance in per-run time reflects the variance in average phenotype size: runs that grow longer expressions spend proportionally more time in the per-example interpreter.

GE is substantially faster than the Assignment 2 SBGP engine (mean 671 s per run) for two reasons. First, the GE phenotype is already represented as a compact binary AST built directly during mapping, whereas A2’s GP engine built a richer `Node` tree carrying structural metadata. Second, derivation-signature similarity is computed on a flat `int[]` via a single tight loop, while A2’s prefix matching on subtree nodes followed an object-graph recursion. Both effects compound when the structural sample size is 50 per individual per generation.

12. Comparison with Assignments 1 and 2

Method	Avg Train MSE	Std. Dev.	Best Train MSE	Avg Test MSE
A1 (Canonical GP)	1.4812×10^{-4}	0.66×10^{-5}	1.4257×10^{-4}	1.7889×10^{-4}
A2 (SBGP, fitness sharing)	1.4611×10^{-4}	0.34×10^{-5}	1.4120×10^{-4}	1.7394×10^{-4}
A3 (SBGE, this work)	1.9314×10^{-4}	2.41×10^{-5}	1.6529×10^{-4}	2.3663×10^{-4}

Table 5: End-to-end comparison across the three assignments. All three were trained on the same 79,994 examples and tested on the same 19,999 examples with identical seeds.

12.1 Quality of solutions

A1 and A2 (both tree-GP) produce noticeably lower training and test MSE than A3 (GE). The A1 canonical GP attains an average training MSE $\sim 23\%$ lower than A3 and the A2 SBGP attains $\sim 24\%$ lower. The test-set ranking is the same: $A2 < A1 < A3$. The best individual produced by A3 (Run 4, train MSE 1.65×10^{-4} , test MSE 1.92×10^{-4}) does, however, come within 16% of the A1 best and outperforms the A1 *average*, showing that GE is capable of finding competitive solutions when its search trajectory survives the single-lag attractor described in Section 10.5. This pattern is consistent with the wider literature: on small symbolic regression problems canonical GE typically underperforms tree-GP because (a) grammar mapping introduces a layer of indirection between genotype and phenotype, so a single codon change can disrupt an arbitrary subtree rather than a single subtree node; and (b) one-point crossover on codons is structurally less informed than subtree crossover.

12.2 Robustness

The standard deviation of training MSE is $\approx 3.7\times$ larger for A3 than for A1 and $\approx 7\times$ larger than for A2. This greater run-to-run variance is again attributable to the codon-to-tree mapping: a small early bias in which production codons appear in the initial population can lock the search onto a particular sub-region of program space, and the linear genome operators are not as effective as subtree crossover at escaping that bias. A2's structural sharing penalty was specifically able to reduce the A1 variance by half because it operated directly on the GP tree; the same penalty in A3 operates one indirection step away and is therefore less effective in absolute terms.

12.3 Speed

A3 is by far the fastest of the three: average 35 s per run versus 298 s for A1 and 671 s for A2. Per the explanation in Section 11, GE pays for this speed with a compact AST representation and an efficient signature-matching routine.

12.4 Summary

The three assignments together produce a clean experimental story. Tree-GP (A1) provides a strong baseline. Adding structural fitness sharing to GP (A2) gives a small but consistent improvement in both mean quality and run-to-run robustness. Replacing the search engine with canonical GE and applying the same structural sharing idea (A3) trades some predictive accuracy for substantially lower wall-clock time and a more compact, interpretable best solution. For the electricity-load regression task the GP family remains the better choice when accuracy is the priority; SBGE is the better choice when training time is the priority or when a grammar-constrained search space is required for downstream interpretability.

13. References

- [1] M. O'Neill and C. Ryan. Grammatical Evolution. *IEEE Transactions on Evolutionary Computation*, 5(4):349–358, 2001.
- [2] R. M. A. Azad and C. Ryan. Sensible Initialisation in Chorus. In *Proc. EuroGP*, pages 394–403, Springer, 2003.
- [3] M. Fenton, J. McDermott, D. Fagan, S. Forstenlechner, M. O'Neill, and E. Hemberg. PonyGE2: Grammatical Evolution in Python. In *GECCO Companion*, pages 1194–1201, ACM, 2017.